14 haftalık yol haritasında 6. hafta

ALGORİTMA VE PROGRAMLAMA – I · DERS NOTU

HAFTA 06

Döngüler I: Tekrarın Mantığı ve Temel Döngü Desenleri

while, do-while ve for; sayaç ve nöbetçi değer; toplayıcıdan basamak ayrıştırmaya döngü desenleri; döngünün makine kodu

MODÜL III · KONTROL AKIŞI · H5–H7

HAFTANIN BÜYÜK SORUSU

Bilgisayar aynı işi milyarlarca kez nasıl yorulmadan yapar ve ne zaman durması gerektiğini nereden bilir?

Dr. Öğr. Üyesi Mehmet Sevri
Recep Tayyip Erdoğan Üniversitesi · Mühendislik ve Mimarlık Fakültesi · Bilgisayar Mühendisliği Bölümü
Bilgisayar Mühendisliği 1. sınıf · 2026–2027 Güz yarıyılı · Sürüm 1.0 · Eylül 2026
`Derleme ortamı: GCC (C17) · gcc -std=c17 -Wall -Wextra -pedantic`

İçindekiler

1. Haftanın Büyük Sorusu ve Öğrenme Çıktıları ... 3
1.1 Öğrenme çıktıları ... 3
2. Geçen Haftadan: Karardan Tekrara ... 4
3. Kavramsal Anlatım ... 4
3.1 Neden döngü? Tekrar, DRY ve döngünün anatomisi ... 4
3.2 Üç döngü: while, do-while ve for ... 5
3.3 Döngüyü denetlemek: sayaç, nöbetçi değer, break ve continue ... 7
3.4 Döngü desenleri: bir kez öğren, her yerde kullan ... 8
3.5 İzleme tablosu, sonlanma ve döngü değişmezi ... 8
3.6 Güvenli girdi döngüsü ve rastgele sayılar ... 9
3.7 Kaputun altında: döngü makine düzeyinde ... 10
4. Çözümlü Örnekler: Örnek Merdiveni ... 11
4.1 ★ basamaklar: üç döngü, toplayıcı, süzme ve sayaç (Örnek 1–4) ... 11
4.2 Çarpım biriktirici: faktöriyel, taşma ve üs (Örnek 5–6) ... 12
4.3 Amiral örnek — Örnek 7 ★★: bilinmeyen sayıda notun ortalaması (nöbetçi değer) ... 14
4.4 Şimdiye kadarki en iyi ve basamaklar (Örnek 8–9) ... 16
4.5 Amiral örnek — Örnek 10 ★★: sayıyı ters çevirme ve palindrom ... 17
4.6 Klasik algoritma, girdi doğrulama ve oyun (Örnek 11–13) ... 19
4.7 Örnek 14 ★★★ — Collatz dizisi ve adım sayısı ... 21
5. Hata Avcısı ... 21
6. Mühendis Gözüyle: Her Döngü Bitmeli ve Doğru Sayıda Dönmeli ... 24
7. Mini-OBS v0.6: N Öğrenci, Akış Hâlinde ... 24
8. Alıştırmalar ve Atölye ... 25
9. Kendini Değerlendirme ... 28
10. AP-2 Köprüsü ve Merak Kutusu ... 28
11. Özet, Kavram Haritası ve Terim Sözlüğü ... 29
12. Kaynaklar ... 30

1. Haftanın Büyük Sorusu ve Öğrenme Çıktıları

Bilgisayar aynı işi milyarlarca kez nasıl yorulmadan yapar ve ne zaman durması gerektiğini nereden bilir?

Şimdiye kadar yazdığımız programlar yukarıdan aşağıya bir kez akıp bitiyordu; H5'te bu akışa yol ayrımları ekledik. Bu hafta programa **geri dönmeyi** öğretiyoruz: aynı deyimleri, bir koşul doğru olduğu sürece yeniden çalıştıran **döngüler** (loop). Bilgisayarın "yorulmadığını" bu makinede ölçtük: 1'den 1 milyara kadar olan sayıları tek tek toplayan en basit döngü, iyileştirmesiz derlendiğinde yaklaşık **2,5 saniyede** bitti (Bölüm 3.7). Ama tekrarın gücü, ne zaman duracağını bilmeyen bir döngüde felakete dönüşebilir.

> GERÇEK DÜNYADAN · **31 Aralık 2008: Zune 30'lar neden açılmadı?**
>
> Microsoft'un 30 GB'lık ilk nesil Zune müzik çalarları 31 Aralık 2008'de açılış ekranında takılıp kaldı. Microsoft, sorunun cihazın iç saat sürücüsünde artık yılın ele alınışıyla ilgili bir hatadan kaynaklandığını açıkladı ve kullanıcılara pilin tamamen bitmesini bekleyip ertesi gün (1 Ocak 2009, GMT öğlen sonrasında) cihazı şarj edip yeniden açmalarını önerdi (Coldewey, 2008; "Microsoft says Zune players working again", 2009).
>
> Sorunlu kod, 1 Ocak 1980'den bu yana geçen gün sayısından yılı hesaplayan bir döngüydü (Hayes, 2009). 2008 artık yıldır; yılın 366. günü olan 31 Aralık'ta `gun` değişkeni tam 366 olur. Koşul `gun > 365` doğru kalır, ama artık yıl dalındaki `gun > 366` yanlıştır ve `else` yoktur: hiçbir değişken değişmez, döngü sonsuza dek döner. Aşağıdaki kesit, döngünün kısaltılmış eğitsel yeniden kurgusudur (özgün kod değildir); tamamını Bölüm 5'te (HA7) çalıştıracağız.
>
> **hata_avcisi/H06_HA7_zune_dongusu.c · satır 20–32**
>
```
20      while (gun > 365) {
21          int artik = (yil % 4 == 0 && yil % 100 != 0) || yil % 400 == 0;
22          if (artik) {
23              if (gun > 366) {
24                  gun -= 366;
25                  yil += 1;
26              }
27              /* gun == 366 ise hiçbir şey değişmez! */
28          } else {
29              gun -= 365;
30              yil += 1;
31          }
32      }
```
> Karşı öykü: anlatılagelen bir öyküye göre küçük Gauss, 1 + 2 + … + 100 toplamını tek tek toplamak yerine uçlardan eşleştirerek (1 + 100, 2 + 99, …) hemen bulmuştur. Öykünün pek çok anlatımı vardır; en eski kaynakta 1–100 sayıları ve eşleştirme yöntemi geçmez (Hayes, 2006). Ders yine de nettir: kimi zaman en iyi döngü, hiç yazılmayan döngüdür (Bölüm 3.7 ve 10).

1.1 Öğrenme çıktıları

ÖÇ 6.1 while, do-while ve for döngülerinin çalışma mantığını ve farklarını açıklar; probleme uygun döngüyü seçer. *(Bölüm 3.1–3.2, Örnek 1)*

ÖÇ 6.2 Sayaç kontrollü ve nöbetçi değer (sentinel) kontrollü döngüler tasarlar. *(Bölüm 3.3, Örnek 4 ve 7)*

ÖÇ 6.3 Biriktirme (toplam/çarpım), sayma, en büyük/en küçük takibi, bayrak ve basamak işleme desenlerini uygular. *(Bölüm 3.4, Örnek 2–11)*

ÖÇ 6.4 İzleme tablosuyla döngü davranışını elle yürütür; sonsuz döngü ve bir-fark (off-by-one) hatalarını tespit eder. *(Bölüm 3.5, 4.3, Bölüm 5)*

2. Geçen Haftadan: Karardan Tekrara

H5'te programın **yol ayrımlarını** kurduk (`if`/`else`, `else if`, `switch`, bileşik koşullar). Bir döngü de "koşul doğruysa gövdeyi çalıştır, sonra **koşula geri dön**" diyen bir karar yapısıdır. Üç geri çağırma sorusu (programlar `kodlar/quiz/` klasöründe):

> **TAHMİN ET · Geri çağırma: üç soru**
>
> - **S1 (switch).** `H06_Q01_switch_duser.c`: `secim = 2`; `case 1` "Bir " yazıp `break` eder, `case 2` "İki " yazar ama `break`'i yoktur, `case 3` "Üç " yazıp `break` eder. Çıktı nedir?
> - **S2 (bileşik koşul).** `puan = 105` için hangi ileti yazılır? `!(puan >= 0 && puan <= 100)` koşulunu `!` kullanmadan yazın.
> - **S3 (girdi doğrulama).** `H06_Q03_tek_sans.c` programına 150 girilirse ne olur? Kullanıcıya yeniden sorabilmek için hangi yapı gerekir?

Yanıtlar (gerçek çıktılar). S1: `İki Üç` — `break` olmadığı için `case 3`'e "düşer" (fall-through); bilinçli düşüşü `/* fall through */` yorumuyla belirtiriz, yoksa GCC `-Wimplicit-fallthrough` ile uyarır. S2: `Geçersiz puan: 105`; De Morgan ile `puan < 0 || puan > 100`. S3: program "Geçersiz puan; program sonlanıyor." yazıp **biter**; kullanıcıya ikinci bir şans verilmez. Yeniden sormak için programın aynı soruyu **tekrar** etmesi gerekir: bu haftanın konusu. S2'deki koşul, Örnek 12'de bir `do-while` döngüsünün koşulu olacak.

3. Kavramsal Anlatım

3.1 Neden döngü? Tekrar, DRY ve döngünün anatomisi

Beş öğrencinin kaydını işleyen bir programı döngüsüz yazarsak aynı satırı beş kez kopyalarız; 500 öğrencide bu yöntem çöker. Yazılım mühendisliğinin **DRY** (Don't Repeat Yourself, kendini tekrar etme) ilkesi, aynı bilginin ya da kalıbın tek bir yerde yazılmasını ister: değişen kısım (burada sıra numarası) bir **değişkene**, tekrar eden kısım bir **döngü gövdesine** dönüşür. Döngünün her bir çalışmasına **tur** ya da **yineleme** (iteration) denir.

H06_K01_dry_tekrar.c · satır 20–23

```
20      /* Döngüyle: kalıp bir kez yazılır, değişen kısım i olur */
21      for (int i = 1; i <= 5; i++) {
22          printf("%d. öğrenci kaydı işlendi\n", i);
23      }
```

Bu üç satır, aynı dosyada kopyala-yapıştırla yazılmış beş `printf` satırıyla birebir aynı çıktıyı üretir (`H06_K01_dry_tekrar.c`). Kopyalanmış sürümde bir değişiklik beş yerde yapılmalıdır; döngüde tek yerde. H2'deki akış diyagramlarında tekrarı, karar kutusundan **geriye dönen bir okla** çizmiştik (H02_07); C'de bu okun üç yazılışı var: `while`, `do-while` ve `for`.

Döngünün dört parçası

Her döngüde, hangi sözdizimiyle yazılırsa yazılsın, dört parça bulunur (Şekil 6.1): (1) **başlatma** — döngü değişkenine ilk değer, yalnızca bir kez; (2) **koşul** — her turun başında (do-while'da sonunda) sınanan ifade; doğruysa tur yapılır, yanlışsa döngüden çıkılır; (3) **gövde** — tekrarlanan iş; (4) **güncelleme** — döngü değişkenini sona doğru yaklaştıran adım. Dördüncü parça unutulursa koşul hiç yanlış olmaz ve **sonsuz döngü** (infinite loop) oluşur (Bölüm 5, HA1).

Şekil 6.1
Döngünün anatomisi: başlatma–koşul–gövde–güncelleme ve n = 3 için çalışma sırası



Şeridi sayarsak önemli bir gözlem çıkar: n tur için koşul **n + 1** kez sınanır; son sınama yanlış çıkar ve döngüyü bitirir. İzleme tablolarında bu son satırı yazmayı unutmak, H2'den beri en sık yapılan hatadır.

3.2 Üç döngü: while, do-while ve for

while: koşul önce

`while (koşul) { gövde }` yapısında koşul **gövdeden önce** sınanır. Koşul doğru olduğu sürece gövde çalışır; koşul baştan yanlışsa gövde **hiç** çalışmaz (0 tur). C'de sıfır olmayan her değer doğru, 0 yanlıştır (H5).

`H06_K02_while_temel.c` 3'ten geriye sayar ("3... 2... 1... Kalkış! (sayac = 0)"); döngüden çıkıldığında `sayac` 0'dır: koşulu yanlış yapan değer döngüden sonra da değişkende kalır. Aynı dosyadaki ikinci döngü `kalan = 0` ile başlar ve gövdesi **hiç** çalışmaz. `while`, "belki hiç tekrar gerekmez" durumları için doğru seçimdir (nöbetçi değerde ilk girdi zaten -1 olabilir, Öklid'de `b` zaten 0 olabilir).

do-while: en az bir kez

`do { gövde } while (koşul);` yapısında gövde **önce** çalışır, koşul **sonra** sınanır; bu yüzden gövde en az bir kez çalışır. Sondaki noktalı virgül zorunludur. Tipik kullanımı, "önce sor, sonra geçerli mi diye bak" düzenidir: girdi doğrulama (Örnek 12, Mini-OBS) ve menü döngüsü (H7).

`H06_K03_do_while_temel.c` farkı tek çalıştırmada gösterir: `i = 10` ve `j = 10` ile başlayan iki döngünün koşulu (`< 5`) baştan yanlıştır; `while` gövdesi hiç çalışmaz, `do-while` gövdesi bir kez çalışır: "do-while gövdesi: j = 10 · Son değerler: i = 10, j = 11".

for: dört parça tek başlıkta

`for (başlatma; koşul; güncelleme) { gövde }` yapısı, while'ın dört parçasını tek satırda toplar; tur sayısının baştan belli olduğu **sayaç kontrollü** döngüler için en okunur yazımdır. Çalışma sırası: başlatma (1 kez) → koşul → gövde → güncelleme → koşul → ... (Şekil 6.1). C99'dan beri sayaç başlıkta bildirilebilir (`for (int i = 1; …)`); böyle bildirilen `i` yalnızca döngünün içinde yaşar (kapsam, H9).

Güncelleme parçası her zaman `i++` olmak zorunda değildir: `i += 2` (iki atlayarak, Örnek 3), `i--` (geri sayım), `i *= 2` (ikiye katlayarak) da geçerlidir. H4'teki `++`, `+=`, `*=` operatörleri burada gerçek işlerine kavuşur.

Üç döngünün birbirine dönüştürülmesi

Örnek 1 (`H06_01_bir_den_ona.c`, aşağıda) aynı işi üç döngüyle yapar. Şekil 6.2, üçünün akış diyagramlarını aynı renk koduyla yan yana gösterir: while ile for'un şeması **aynıdır**; do-while'da karar kutusu gövdeden sonraya geçer. Dönüşüm kuralı: for'un başlatması döngüden önceye, güncellemesi gövdenin sonuna taşınırsa while elde edilir. while'dan do-while'a geçerken ise ilk turda koşulun doğru olduğu bilinmelidir; aksi hâlde davranış değişir (K03 programı). `H06_K04_for_while_donusum.c` aynı dönüşümü sayacın kapsamıyla birlikte gösterir: for başlığında bildirilen `i` döngüden sonra yoktur, while için dışarıda bildirilen `k` ise döngüden sonra 4 değeriyle yaşar.

H06_01_bir_den_ona.c · satır 14–32

```
14      int i = 1;                  /* başlatma   */
15      while (i <= 10) {           /* koşul      */
16          printf("%d ", i);       /* gövde      */
17          i++;                    /* güncelleme */
18      }
19      printf("\n");
20
21      /* 2) do-while: gövde önce çalışır, koşul SONRA sınanır */
22      printf("do-while: ");
23      int j = 1;
24      do {
25          printf("%d ", j);
26          j++;
27      } while (j <= 10);
28      printf("\n");
29
30      /* 3) for: başlatma; koşul; güncelleme tek satırda */
31      printf("for     : ");
32      for (int k = 1; k <= 10; k++) {
```

ÇIKTI

```
$ ./bir_den_ona
while   : 1 2 3 4 5 6 7 8 9 10
do-while: 1 2 3 4 5 6 7 8 9 10
for     : 1 2 3 4 5 6 7 8 9 10
```

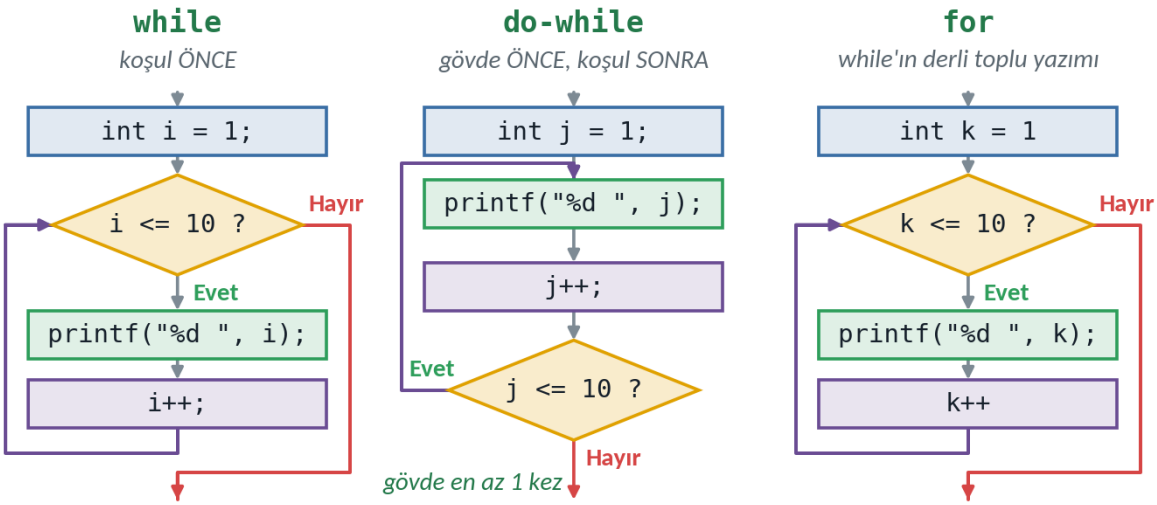
Tablo 6.1
Üç döngünün karşılaştırılması

	while	do-while	for
Koşul ne zaman sınanır?	Gövdeden önce	Gövdeden sonra	Gövdeden önce
En az tur sayısı	0	1	0
Başlatma nerede?	Döngüden önce	Döngüden önce	Başlıkta (1. parça)
Güncelleme nerede?	Gövdenin içinde	Gövdenin içinde	Başlıkta (3. parça)
Tipik kullanım	Nöbetçi değer, Öklid, basamak	Girdi doğrulama, menü	Sayaç kontrollü: 1..N, tablo

Hangisini seçmeli? İki soru çoğu durumda yeterlidir: **Tur sayısı döngüye girmeden belli mi?** Evetse `for`. Değilse: **Gövde en az bir kez çalışmalı mı?** Evetse `do-while`, değilse `while`. Üçü de birbirine

dönüştürülebildiği için seçim bir **okunabilirlik** kararıdır: okuyan kişiye niyetinizi en açık gösteren yazımı seçin.

Şekil 6.2
while, do-while ve for akış diyagramları (ISO 5807) yan yana



3.3 Döngüyü denetlemek: sayaç, nöbetçi değer, break ve continue

Bir döngünün **ne zaman duracağı** iki temel biçimde belirlenir. **Sayaç kontrollü** (counter-controlled) döngüde tur sayısı döngüye girmeden bellidir: “10 kez”, “N öğrenci için”. **Nöbetçi değer kontrollü** (sentinel-controlled) döngüde tur sayısı bilinmez; veri, “bitti” anlamına gelen ve geçerli bir veri olamayacak özel bir değerle, **nöbetçi değerle** (sentinel value) sonlandırılır: not girişinde -1, isim listesinde boş satır. Örnek 7’nin akış diyagramı ve Alıştırma 6.4 bu deseni işler.

Nöbetçi döngünün klasik yazımı **öncül okumadır** (priming read): ilk değer döngüden önce okunur; gövde değeri işledikten sonra bir sonrakini okur. Böylece koşul her zaman “son okunan değer nöbetçi mi?” sorusunu sorar. Üçüncü bir denetim biçimi de **olay/koşul kontrollü** döngülerdir: “kullanıcı doğru tahmini yapana kadar” (Örnek 13), “kalan sıfır olana kadar” (Örnek 11).

break ve continue

`break`, içinde bulunduğu döngüyü **hemen** bitirir; akış döngüden sonraki deyimle sürer (H5’te `switch`’ten de çıkmıştık). `continue`, yalnızca **o turun kalanını** atlar: while/do-while’da koşula, for’da **güncellemeye** gider. Aramada “bulundu, daha fazla bakmaya gerek yok” durumunda `break`, “bu öge işlenmeyecek” durumunda `continue` doğaldır. `H06_K05_break_continue.c`’nin ilk iki döngüsü ikisini ayrı ayrı (50’den sonraki ilk 7 katı: 56; 3’ün katları atlanarak 1..10), üçüncüsü birlikte kullanır:

H06_K05_break_continue.c · satır 29–38

```
29      /* birlikte: 3'ün katlarını atla, 8'e gelince dur */
30      for (int i = 1; i <= 10; i++) {
31          if (i % 3 == 0) {
32              continue;
33          }
34          if (i == 8) {
35              break;
36          }
37          printf("%d ", i);
38      }
```

```
ÇIKTI
$ ./break_continue
50'den sonraki ilk 7 katı: 56
1 2 4 5 7 8 10
1 2 4 5 7
```

Ölçülü kullanın: birden çok `break`/`continue` içeren uzun bir gövde, döngünün ne zaman bittiğini koşula bakarak anlamayı zorlaştırır. İç içe döngülerde `break`'in yalnızca **en içteki** döngüden çıktığını H7'de göreceğiz.

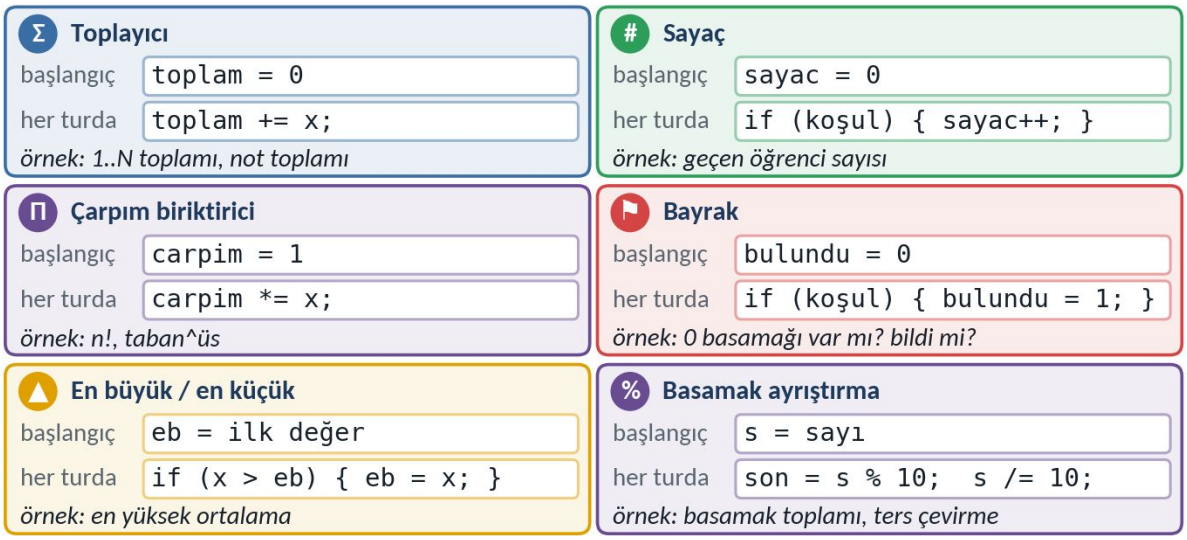
3.4 Döngü desenleri: bir kez öğren, her yerde kullan

Döngülerle çözülen problemlerin büyük çoğunluğu birkaç tekrarlayan kalıba, **desene** dayanır (Şekil 6.3). Her desenin iki bileşeni vardır: döngüden **önce** doğru başlangıç değeri ve **her turda** aynı güncelleme. Başlangıç değeri, işlemin **etkisiz elemanıdır**: toplamada 0, çarpmada 1; en büyük/en küçük için ise **ilk değer** (Örnek 8).

Toplayıcı (accumulator; Örnek 2, 7) ve **sayaç** (counter; Örnek 3, Mini-OBS) 0 ile, **çarpım biriktirici** (Örnek 5, 6) 1 ile başlar; çarpım 0 ile başlatılırsa sonuç hep 0 olur. **Bayrak** (flag) "en az bir tane var mı?" sorusunu yanıtlar ve genellikle `break` ile birlikte kullanılır (Örnek 13). **En büyük/en küçük** adayı ilk değerle başlar, daha iyisi gelince güncellenir (Örnek 8). **Basamak ayrıştırmada** `sayi % 10` son basamağı alır, `sayi /= 10` son basamağı atar (Örnek 9, 10; H4'teki %10 ve /10 deseninin döngülü hâli).

Altı desenin en yalın, birlikte çalışan hâlleri `H06_K06_desenler.c` dosyasındadır; gerçek çıktısı: `toplam = 21, sayac = 2, carpim = 720; sifir_var = 1, en_buyuk = 8`.

Şekil 6.3
Altı döngü deseni: başlangıç değeri, her turdaki güncelleme ve örnek kullanım



3.5 İzleme tablosu, sonlanma ve döngü değişmezi

Bir döngüyü anlamanın en güvenilir yolu, H2'den bildiğimiz **izleme tablosudur** (trace table): her koşul denetiminde değişkenlerin değerini bir satıra yazarız. `H06_K20_izle_toplam.c` (n = 5) için Tablo 6.2; son satırda koşul yanlıştır ve döngü biter. Webdeki "Döngü izleyici" simülasyonu aynı tabloyu satır satır oluşturur.

Tablo 6.2
H06_K20 (n = 5) izleme tablosu: her satır bir koşul denetimi

Denetim	i	toplam (önce)	koşul	gövde
1	1	0	1 <= 5 → doğru	toplam = 0 + 1 = 1
2	2	1	2 <= 5 → doğru	toplam = 1 + 2 = 3
3	3	3	3 <= 5 → doğru	toplam = 3 + 3 = 6
4	4	6	4 <= 5 → doğru	toplam = 6 + 4 = 10
5	5	10	5 <= 5 → doğru	toplam = 10 + 5 = 15
6	6	15	**6 <= 5 → yanlış**	döngüden çık

Döngü ne zaman biter? İlerleme ölçüsü

Bir döngünün **bittiğinden** emin olmak için her turda kesin olarak küçülen ve alttan sınırlı bir nicelik ararız; buna sezgisel olarak **ilerleme ölçüsü** diyelim. `for (i = 1; i <= n; i++)` için `n - i + 1` her turda 1 azalır ve 0'a inince döngü biter. Öklid'de `y`, basamak döngüsünde `sayi` her turda kesin küçülür (Örnek 9–11). Zune döngüsünde ise artık yılın 366. gününde **hiçbir şey küçülmez**; Collatz döngüsünde (Örnek 14) böyle bir ölçü bilinmiyor.

Döngü değişmezi: neden doğru?

Döngünün **doğru sonucu** verdiğini gerekçelendirmek için her turun başında doğru olan bir önerme kurarız: **döngü değişmezi** (loop invariant). Toplam döngüsü için: "her turun başında `toplam = 1 + 2 + … + (i − 1)`". İlk turda `i = 1`, toplam 0'dır (boş toplam): önerme doğru. Gövde `i`'yi ekler, güncelleme `i`'yi bir artırır: önerme bir sonraki turun başında da doğru. Döngü `i = n + 1` iken bitince önerme "`toplam = 1 + … + n`" der: istediğimiz sonuç. `H06_K08_degismez.c` önermeyi her turun başında `(i - 1) * i / 2` ile karşılaştırarak gerçekten denetler; N = 5 için beş turun hepsinde "korunuyor" yazar ve çıkışta 15 = N(N+1)/2 bulur.

3.6 Güvenli girdi döngüsü ve rastgele sayılar

scanf'in dönüş değeri ve giriş tamponu

H3'te gördük: `scanf` başarıyla okuduğu öge sayısını döndürür: `scanf("%d", &x)` için 1 (okundu), 0 (sayı yerine başka karakter), `EOF` (girdi kapandı: Linux'ta Ctrl+D, Windows'ta Ctrl+Z ve Enter). 0 döndüğünde sorunlu karakterler **tamponda kalır** ve okuma konumu ilerlemez; dönüş değerine bakmayan bir döngü aynı karakterlere tekrar tekrar takılır (HA6).

Kaynaklarda bu temizleme döngüsü tek satırda, `;` gövdeyle yazılır; HA2'deki tuzağa benzediği için biz süslü parantezli yazıyoruz. Bu haftadan itibaren her `scanf` çağrısının **dönüş değeri denetlenir** (yol haritası Bölüm 6.5).

Rastgele sayılar: rand ve srand

`<stdlib.h>`'deki `rand()`, 0 ile `RAND_MAX` arasında bir tamsayı döndürür; C standardı yalnızca `RAND_MAX` ≥ 32767 olmasını ister. Bu ortamda (glibc) `RAND_MAX = 2 147 483 647`; Windows'ta Microsoft'un C kitaplığında 32767'dir. `rand() % 100` 0..99 aralığını, `rand() % 100 + 1` ise 1..100 aralığını verir. Üretilen sayılar **sözde rastgeledir** (pseudo-random): bir başlangıç değerinden, **tohumdan** (seed) belirlenimci bir formülle hesaplanır. Aynı tohum aynı diziyi üretir; `H06_K12_rand_srand.c`'de `srand(2026)` ardından beş kez `rand() % 100 + 1` bu makinede her

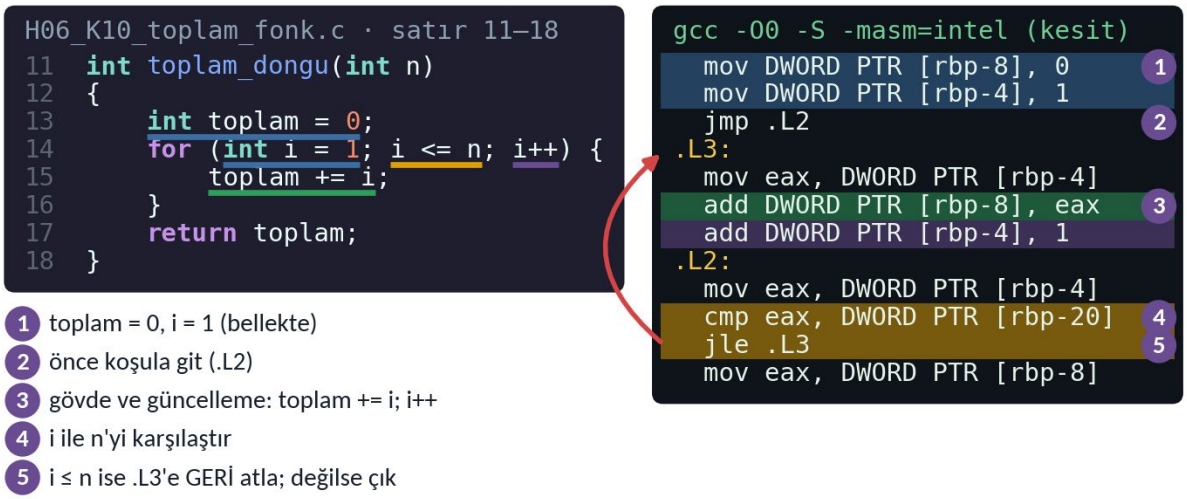
seferinde `38 73 24 70 87` verir; son satırdaki saat tohumlu değer ise iki çalıştırmada farklı çıktı (73 ve 90).

Her çalıştırmada farklı bir dizi için tohum, saatten alınır: `srand((unsigned)time(NULL));` (`<time.h>`). `srand` programın başında **bir kez** çağrılır; döngünün içinde çağırmak aynı saniyede hep aynı sayıyı verir, çünkü `time` saniye çözünürlüklüdür. Kayıtlı çıktıların yeniden üretilebilir olması için Örnek 13'ü `-DSABIT_TOHUM=2026` ile derledik (Bölüm 4.6, Örnek 13).

3.7 Kaputun altında: döngü makine düzeyinde

> KAPUTUN ALTINDA · **Döngü = geriye doğru koşullu atlama**
>
> İşlemcinin "döngü" diye bir komutu yoktur; yalnızca karşılaştırma (`cmp`) ve atlama komutları vardır. H5'te `if`'in ileriye doğru koşullu atlamaya dönüştüğünü gördük; döngü ise **geriye doğru koşullu atlamadır**. `H06_K10_toplam_fonk.c`'deki toplam döngüsünü GCC ile iyileştirmesiz (`-O0`) derleyip ürettiği assembly'yi okuyalım (Şekil 6.4, gerçek çıktı `kodlar/ciktilar/H06_K10_toplam_fonk_gcc_O0_asm.txt`).
>
> Derleyici önce **koşula** atlar (`jmp .L2`), koşul `cmp` + `jle .L3` ile sınanır ve doğruysa gövdenin başındaki `.L3` etiketine **geri** atlanır; yanlışsa bir alt satıra düşülür ve döngü biter. `-O0`'da `i` ve `toplam` her turda bellekten (`[rbp-4]`, `[rbp-8]`) okunup yazılır.
>
> Bu hafta assembly'yi **Intel sözdizimiyle** (`-masm=intel`) okuyoruz: hedef önce, kaynak sonra yazılır; yazmaç ve sabitlerde önek yoktur (`mov DWORD PTR [rbp-8], 0`). H5'teki **AT&T** sözdiziminde (GCC'nin varsayılanı) işlenen sırası terstir, yazmaçlar `%`, sabitler `$` ile yazılır: aynı satır `movl $0, -8(%rbp)` (gerçek çıktı, `kodlar/ciktilar/H06_K10_toplam_fonk_gcc_O0_att_asm.txt`).

Şekil 6.4
C ↔ assembly (gcc -O0): koşula atla, gövde, güncelleme, geriye koşullu atlama



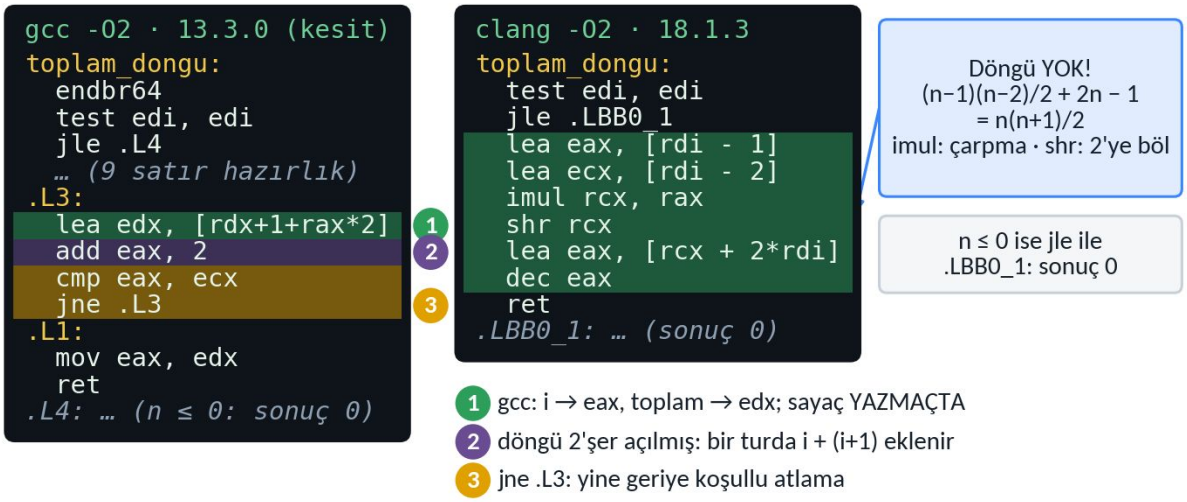
İyileştirme: sayaç yazmaçta, döngü formülde

Aynı fonksiyonu `-O2` ile derleyince iki derleyici farklı yollar seçer (Şekil 6.5). **GCC 13** döngüyü korur ama `i`'yi `eax`, `toplam`'ı `edx` **yazmacında** tutar (bellek erişimi yok) ve döngüyü **2'şer açar**: bir turda `lea edx, [rdx+1+rax*2]` ile `i + (i + 1)` ekler. **Clang 18** ise döngüyü tamamen kaldırır; yerine `(n−1)(n−2)/2 + 2n − 1` hesaplayan birkaç komut koyar; bu ifade cebirsel olarak `n(n+1)/2`'dir, yani Gauss'un formülü. Derleyici döngünün ne hesapladığını tanımış ve **kapalı formüle** (closed form) çevirmiştir.

Etkisini ölçtük. `H06_K09_milyar_tur.c`, N'yi klavyeden okuyup 1 + … + N'yi `unsigned long long` ile toplar. N = 10^9 (bir milyar) için `time` komutuyla ölçülen süreler gcc -O0 ile **2,495 s**, gcc -O2 ile **0,334 s**, clang -O2 ile **0,002 s** oldu: aynı kaynak kod gcc -O2 ile yaklaşık 7 kat hızlandı; clang -O2'nin süresi ise

sürecin başlatılma süresine yakın, yani ölçüm sınırında: hızlanma **binlerce kat mertebesindedir**. N klavyeden okunduğu için derleyici sonucu önceden hesaplayamaz; hız farkı gerçekten üretilen koddan gelir. Ölçüm kaydı: `kodlar/ciktilar/H06_K09_milyar_tur.txt`.

Şekil 6.5
gcc -O2 ile clang -O2'nin gerçek çıktıları: yazmaçta açılmış döngü ve kapalı formül



4. Çözümlü Örnekler: Örnek Merdiveni

Örnekler kolaydan zora sıralıdır ve her basamak bir yeni öge ekler (dosyalar `kodlar/H06_01_…` – `H06_14_…`). Her biri `gcc -std=c17 -Wall -Wextra -pedantic -g` ile **sıfır uyarıyla** derlenir, AddressSanitizer ve UBSan ile de temiz çalışır. Çıktılar `kodlar/ciktilar/` altındaki gerçek çalıştırmalardır. Örnek 7 ve 10, yol haritasındaki standart çözümlü örnek şablonunun tamamıyla işlenmiştir.

4.1 ★ basamaklar: üç döngü, toplayıcı, süzme ve sayaç (Örnek 1–4)

Örnek 1 ★ — 1'den 10'a üç döngüyle

Problem. 1'den 10'a kadar sayıları üç döngü türüyle ayrı satırlara yazdırın. **Algoritma:** Şekil 6.2.

Kod ve gerçek çıktı Bölüm 3.2'de incelendi; dosyanın kalanında aynı iş `for (int k = 1; k <= 10; k++)` ile yapılır. **Değiştir.** (a) Üç döngüyü 10'dan 1'e geri sayacak biçimde değiştirin. (b) Başlangıç değerlerini 11 yapın: hangi döngü yine de bir sayı yazar, neden? (c) `k`'yi döngüden sonra yazdırmayı deneyin; derleyici ne der?

Örnek 2 ★ — 1'den N'e kadar toplam (toplayıcı)

Problem. N okunur, 1+2+…+N yazdırılır. H2'de (H02_07) aynı algoritmayı `while` ile yazmıştık; bu kez `for` ve `+=` ile, `scanf` denetimiyle yazıyoruz. Toplam `int` olduğundan N ≤ 65 535 olmalıdır: 65 536 için n(n+1)/2 = 2 147 516 416 > `INT_MAX` ve işaretli taşma tanımsız davranıştır. Program bu sınırı `EN_BUYUK_N` sabitiyle **işlemden önce** denetler.

H06_02_toplam_n.c · satır 26–31

```
26      int toplam = 0;                 /* toplayıcı: etkisiz eleman 0 */
27      for (int i = 1; i <= n; i++) {
28          toplam += i;                /* toplam = toplam + i */
29      }
30
31      printf("Toplam (1..%d) = %d\n", n, toplam);
```

Gerçek çıktılar ve sınır durumu. N = 5 için "Toplam (1..5) = 15". N = 0 ve N < 0 için koşul ilk denetimde yanlış olur, döngü hiç çalışmaz ve toplamın başlangıç değeri yazılır (N = 0 için gerçek çıktı: "Toplam (1..0) = 0"): boş toplam 0'dır. Sığan en büyük N için "Toplam (1..65535) = 2147450880", 65 536 için "N en fazla 65535 olabilir." (gerçek çıktılar).

Örnek 3 ★ — Çift sayıların toplamı: iki yol

Problem. 1..N arasındaki çift sayıların toplamı. Yol A her sayıyı dolaşıp `if (i % 2 == 0)` ile süzer; Yol B `i = 2`'den başlayıp `i += 2` ile yalnızca çiftleri üretir. İkisi aynı sonucu verir ama tur sayıları farklıdır (`H06_03_cift_toplam.c`):

ÇIKTI

```
$ ./cift_toplam
N değerini girin: 10
Yol A (if ile süzme): toplam = 30, 10 tur
Yol B (adım 2)      : toplam = 30, 5 tur
```

Yol B, işi yarı tur sayısıyla yapar. Genel ders: **gereksiz turları süzmek yerine hiç üretmemek** daha verimlidir (AP-2'de büyüme hızı tartışmasının ilk tohumu). Değiştir: 1..N arasında 3'ün katlarının toplamını iki yolla yazın.

Örnek 4 ★ — Çarpım tablosu (sayaç kontrollü)

H06_04_carpim_tablosu.c · satır 19–22

```
19      printf("--- %d için çarpım tablosu ---\n", sayi);
20      for (int i = 1; i <= 10; i++) {          /* sayaç kontrollü: tam 10 tur */
21          printf("%2d x %2d = %4d\n", sayi, i, sayi * i);
22      }
```

Gerçek çıktı (7 için): "--- 7 için çarpım tablosu ---" başlığının altında `7 x  1 =    7` … `7 x 10 =   70` satırları (`kodlar/ciktilar/`). Tur sayısı (10) baştan bellidir: tipik bir sayaç kontrollü döngü. `%2d` ve `%4d` alan genişlikleri (H3) sütunları hizalar. 10 × 10'luk tablonun tamamı H7'de iç içe döngüyle gelecek.

4.2 Çarpım biriktirici: faktöriyel, taşma ve üs (Örnek 5–6)

Örnek 5 ★★ — Faktöriyel ve taşma

Problem. n! = 1 · 2 · … · n değerlerini n = 1..21 için yazdırın; her birinin `int`'e sığıp sığmadığını ve 64 bitlik `unsigned long long`'un nerede yetmediğini gösterin. H2'deki faktöriyel programına (H02_10) "0 ≤ n ≤ 12" kısıtı koymuştuk; nedenini şimdi görüyoruz.

H06_05_faktoriyel_tasma.c · satır 14–24

```c
    unsigned long long carpim = 1;      /* çarpım biriktirici: 1 ile başlar */

    printf(" n  %-22s %s\n", "n! (unsigned long long)", "int'e sığar mı?");
    for (int n = 1; n <= 21; n++) {
        /* Taşma denetimi çarpmadan ÖNCE: carpim * n, ULLONG_MAX'ı aşar mı? */
        int tasacak = carpim > ULLONG_MAX / (unsigned long long)n;
        carpim *= n;                    /* işaretsiz: 2^64 modunda sarar */
        printf("%2d  %-22llu %s%s\n", n, carpim,
               carpim <= INT_MAX ? "evet" : "hayır",
               tasacak ? "   <- 64 bite sığmadı: sarmış değer!" : "");
    }
```

ÇIKTI (GERÇEK; N = 1..10 VE 14..18 SATIRLARI ÇIKARILDI)

```
 n  n! (unsigned long long) int'e sığar mı?
11  39916800               evet
12  479001600              evet
13  6227020800             hayır
…
19  121645100408832000     hayır
20  2432902008176640000    hayır
21  14197454024290336768   hayır   <- 64 bite sığmadı: sarmış değer!
```

Satır satır. Çarpım biriktirici **1** ile başlar. Taşma, çarpmadan **önce** sezilir: `carpim > ULLONG_MAX / n` ise `carpim * n` 64 bite sığmayacaktır. 20! = 2 432 902 008 176 640 000 sığan son değerdir; 21!'in gerçek değeri 51 090 942 171 709 440 000'dir ve $2^{64} \approx 1{,}84 \times 10^{19}$'u aşar. **İşaretsiz** tamsayıda bu durum C'de **tanımlıdır**: sonuç 2^{64} modunda sarar, program 14 197 454 024 290 336 768 yazar. 13! = 6 227 020 800 ise `INT_MAX`'ı (2 147 483 647) aşar (sunumdaki logaritmik büyüme grafiği).

> **HATA AVCISI · int ile 13!: tanımsız davranış**
>
> Aynı çarpımı `int` ile yapmak (`H06_K07_faktoriyel_int_ub.c`, BU DOSYA KASITLI TANIMSIZ DAVRANIŞ İÇERİR) derleyici uyarısı üretmez, çünkü değerler çalışma anında oluşur. **İşaretli tamsayı taşması tanımsız davranıştır** (H3): standart, programın ne yapacağını hiç belirlemez; "şu değer çıkar" diye bir sonuç yoktur. Döngü `int carpim = 1;` ile başlar ve `carpim *= n;` yapar; sorunu çalışma anında yakalamak için programı `-fsanitize=undefined` ile derleriz.
>
> UBSan'ın gerçek iletisi: `H06_K07_faktoriyel_int_ub.c:15:16: runtime error: signed integer overflow: 479001600 * 13 cannot be represented in type 'int'`. İletiden sonra yazılan 13! satırı güvenilir bir değer değildir. Çözüm: yeterince geniş bir tip seçmek (Örnek 5) ya da çarpmadan önce sınırı denetlemek.

Örnek 6 ★★ — Tekrarlı çarpma ile üs alma

`H06_06_us_alma.c` çarpım biriktiricisini `long long sonuc = 1;` ile başlatır ve `for (int i = 1; i <= us; i++)` döngüsünde `sonuc *= taban;` yapar. Gerçek çıktılar: "2^10 = 1024 (10 çarpma)", "-2^3 = -8 (3 çarpma)", "7^0 = 1 (0 çarpma)".

`us` = 0 iken koşul ilk denetimde yanlıştır, döngü hiç çalışmaz ve sonuç başlangıç değeri 1'dir: $x^0 = 1$ tanımı kendiliğinden sağlanır. Negatif üs `scanf` denetiminde reddedilir. `math.h`'deki `pow` (H4) `double` döndürür; tamsayı üsler için döngü kesin sonuç verir. Değiştir: sonuç `long long`'a sığmayacaksa çarpmadan önce uyarı veren bir denetim ekleyin (Örnek 5'teki gibi).

4.3 Amiral örnek — Örnek 7 ★★: bilinmeyen sayıda notun ortalaması (nöbetçi değer)

1. Problem. Bir öğretim üyesi sınav notlarını teker teker girer; kaç not gireceği önceden belli değildir. Girilen notların sayısını, toplamını ve ortalamasını (2 ondalık) yazdıran; bitişi **-1** ile belirten bir program yazın.

Tablo 6.3
Örnek 7'nin girdi–işlem–çıktı çözümlemesi

GİRDİ	İŞLEM	ÇIKTI
• Notlar (tamsayı, bilinmeyen sayıda) • Nöbetçi değer: -1	• Toplayıcı: toplam += not • Sayaç: sayac++ • ortalama = toplam / sayac (kesirli)	• Not sayısı, toplam, ortalama • Hiç not yoksa uyarı

Kısıtlar ve sınır durumları. İlk girdi -1 ise sayac = 0'dır ve **sıfıra bölme** yapılmamalıdır. Sayı olmayan bir girdi (`abc`) döngüyü bitirir: `scanf` 1 döndürmediği için koşulun ilk yarısı yanlıştır. **3. Algoritma:** Şekil 6.6 (öncül okuma).

4. C kodu.

H06_07_nobetci_ortalama.c · satır 9–34

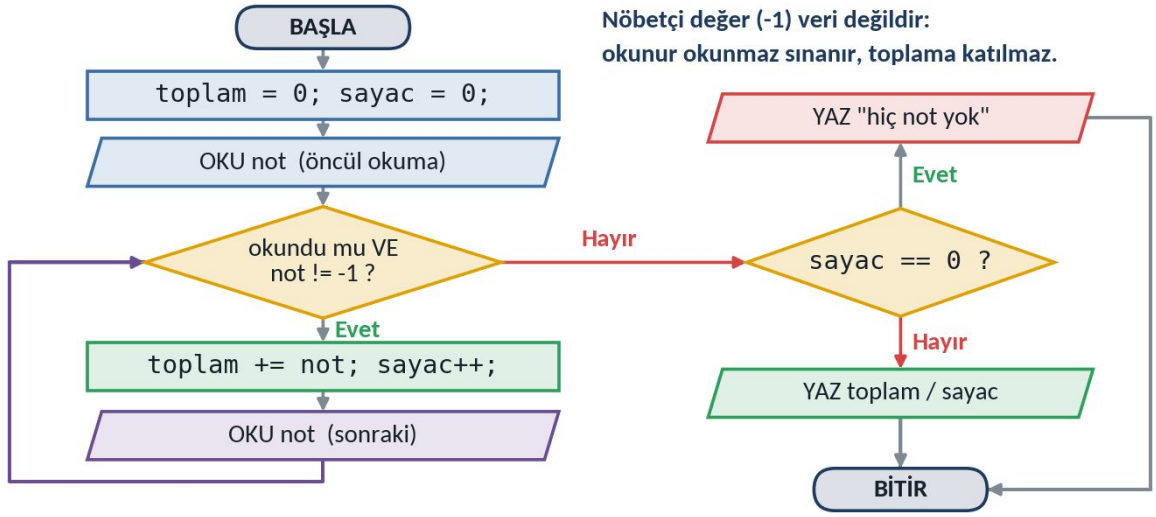
```c
#include <stdio.h>

int main(void)
{
    const int NOBETCI = -1;            /* "girdi bitti" işareti */
    int not_degeri;
    int toplam = 0;                    /* toplayıcı */
    int sayac = 0;                     /* sayaç     */

    /* öncül okuma: ilk değer döngüden ÖNCE istenir */
    printf("Not 1 (bitirmek için %d): ", NOBETCI);
    while (scanf("%d", &not_degeri) == 1 && not_degeri != NOBETCI) {
        toplam += not_degeri;
        sayac++;
        printf("Not %d (bitirmek için %d): ", sayac + 1, NOBETCI);
    }

    if (sayac == 0) {
        printf("Hiç not girilmedi; ortalama hesaplanamaz.\n");
    } else {
        double ortalama = (double)toplam / sayac;
        printf("%d not, toplam = %d, ortalama = %.2f\n",
               sayac, toplam, ortalama);
    }
    return 0;
}
```

Şekil 6.6
Nöbetçi değer kontrollü döngü ve öncül okuma (Örnek 7'nin akış diyagramı)



5. Satır satır. Satır 13: nöbetçi değer adlandırılmış bir sabittir (sihirli sayı yok). Satır 19: öncül okumanın istemi döngüden önce. Satır 20: okuma başarılı mı **ve** değer nöbetçi değil mi? `&&` kısa devre yapar: okuma başarısızsa `not_degeri` hiç sınanmaz (H4). Satır 23: bir sonraki notun istemi gövdenin sonunda. Satır 29: `(double)` dönüşümü tamsayı bölmesini önler (H4).

Tablo 6.4
Örnek 7 izleme tablosu (girdi: 70, 85, 90, -1)

Okuma	not_degeri	koşul	toplam	sayac
1	70	doğru	70	1
2	85	doğru	155	2
3	90	doğru	245	3
4	-1	**yanlış (nöbetçi)**	245	3

7–8. Örnek çalıştırma ve test tablosu (gerçek çıktılar):

```
ÇIKTI
$ ./nobetci_ortalama      # normal
Not 1 (bitirmek için -1): 70
Not 2 (bitirmek için -1): 85
Not 3 (bitirmek için -1): 90
Not 4 (bitirmek için -1): -1
3 not, toplam = 245, ortalama = 81.67
```

Tablo 6.5
Örnek 7 test tablosu

Test	Girdi	Beklenen	Gerçek çıktı (son satır)	Sonuç
Normal	70 85 90 -1	3 not, ortalama 81.67	3 not, toplam = 245, ortalama = 81.67	✓
Sınır: hiç not	-1	Uyarı, bölme yok	Hiç not girilmedi; ortalama hesaplanamaz.	✓
Hatalı girdi	55 abc	Döngü biter, 1 not	1 not, toplam = 55, ortalama = 55.00	✓

9. Değiştir–genişlet. (a) -1 dışındaki geçersiz notları (0–100 dışı) atlayın: hangi anahtar sözcük? (`continue`). (b) En yüksek notu da bulun (Örnek 8'in deseni). (c) "abc" girildiğinde döngüyü bitirmek yerine uyarı verip devam edin (Örnek 12'nin tampon temizleme deseni).

4.4 Şimdiye kadarki en iyi ve basamaklar (Örnek 8–9)

Örnek 8 ★★ — Girilen sayıların en büyüğü ve en küçüğü

Problem. N sayı okunur; en büyüğü ve en küçüğü, **kaçıncı sırada** geldikleriyle birlikte yazdırılır. H2'deki H02_09'un iki yönlü ve `for`'lu sürümü.

H06_08_en_buyuk_kucuk.c · satır 25–42

```c
    int en_buyuk = x, buyuk_sira = 1;   /* ilk sayı: şimdiye kadarki en iyi */
    int en_kucuk = x, kucuk_sira = 1;

    for (int i = 2; i <= n; i++) {
        printf("%d. sayı: ", i);
        if (scanf("%d", &x) != 1) {
            printf("Geçersiz girdi.\n");
            return 1;
        }
        if (x > en_buyuk) {             /* daha iyisi geldi: güncelle */
            en_buyuk = x;
            buyuk_sira = i;
        }
        if (x < en_kucuk) {
            en_kucuk = x;
            kucuk_sira = i;
        }
    }
```

ÇIKTI

```
$ ./en_buyuk_kucuk      # hepsi negatif
Kaç sayı girilecek? 3
1. sayı: -5
2. sayı: -2
3. sayı: -9
En büyük: -2 (2. sayı)
En küçük: -9 (3. sayı)
```

Neden ilk sayıyla başlatıyoruz? Adayı 0 ile başlatsaydık, hepsi negatif olan bu testte en büyük yanlışlıkla 0 çıkardı; gerçek çıktı -2'dir. Güvenli seçenekler: ilk değer ya da `<limits.h>`'deki `INT_MIN`/`INT_MAX`. `>` yerine `>=` yazılsaydı eşit değerlerde **son** gelenin sırası kaydedilirdi. Bu "şimdiye kadarki en iyi" deseni, AP-2'de seçme sıralamasının iç döngüsüdür.

Örnek 9 ★★ — Basamak sayısı ve basamak toplamı

H06_09_basamak_islemleri.c · satır 20–30

```c
    long long sayi = n;                 /* long long: -INT_MIN de sığar */
    if (sayi < 0) {
        sayi = -sayi;                   /* işareti at: basamaklar aynı kalır */
    }

    int basamak_sayisi = 0, basamak_toplami = 0;
    do {                                /* en az bir tur: 0 -> 1 basamak */
        basamak_toplami += sayi % 10;   /* son basamağı al */
        basamak_sayisi++;
        sayi /= 10;                     /* son basamağı at */
    } while (sayi != 0);
```

Gerçek çıktılar: “2026: 4 basamak, basamak toplamı 10”, “0: 1 basamak, basamak toplamı 0”, “-2147483648: 10 basamak, basamak toplamı 47”.

Neden do-while? 0 girildiğinde de 1 basamak saymak gerekir; `while (sayi != 0)` gövdesi 0 için hiç çalışmazdı. H02_11 aynı sorunu `while (sayi >= 10)` ve `basamak = 1` başlangıcıyla çözmüştü; iki çözüm eşdeğerdir. **Neden long long?** `-INT_MIN`, yani 2 147 483 648 `int`’e sığmaz; `sayi = -sayi` `int` ile yapılsaydı tanımsız davranış olurdu. Son test tam bu sınırı dener. **İlerleme ölçüsü:** `sayi /= 10` her turda sayıyı en az bir basamak kısaltır; döngü en çok 10 tur sürer.

4.5 Amiral örnek — Örnek 10 ★★: sayıyı ters çevirme ve palindrom

1. Problem. Negatif olmayan bir tamsayıyı basamaklarından ters çevirip yazdırın ve sayının **palindrom** (tersten de aynı okunan: 12321) olup olmadığını söyleyin. Kimlik ve kart numaralarındaki denetim basamakları, sayısal kodlarda yansıma denetimleri bu tür basamak işlemleriyle yapılır.

2. Çözümleme. Girdi: n ≥ 0 tamsayı. İşlem: `sayi > 0` oldukça son basamağı al, `ters = ters * 10 + son`, son basamağı at; sonunda `ters == n` karşılaştırması. Çıktı: ters sayı ve palindrom kararı. Negatif girdi reddedilir. 1 999 999 999’un tersi 9 999 999 991’dir ve `int`’e sığmaz: `ters` bu yüzden `long long`. Sondaki sıfırlar kaybolur: 1230’un tersi “0321” değil 321’dir. **3. Algoritma.** Her turda üç adım: son basamağı **al**, tersin sağına **ekle** (`ters * 10` sağda yer açar), sayıdan **at** (izleme tablosu aşağıda).

4. C kodu.

H06_10_ters_palindrom.c · satır 8–34

```c
#include <stdio.h>

int main(void)
{
    int n;
    printf("Negatif olmayan bir tamsayı girin: ");
    if (scanf("%d", &n) != 1 || n < 0) {
        printf("Geçersiz girdi.\n");
        return 1;
    }

    int sayi = n;
    long long ters = 0;                 /* 1999999999'un tersi int'e sığmaz */
    while (sayi > 0) {
        int son = sayi % 10;            /* 1) son basamağı al        */
        ters = ters * 10 + son;         /* 2) tersin sağına ekle     */
```

```c
        sayi /= 10;                     /* 3) son basamağı at        */
    }

    printf("Ters: %lld\n", ters);
    if (ters == n) {
        printf("%d bir palindrom sayıdır.\n", n);
    } else {
        printf("%d palindrom değildir.\n", n);
    }
    return 0;
}
```

5. Satır satır. Satır 19: girdiyi korumak için kopya üzerinde çalışırız; döngü `sayi`'yi 0'a kadar tüketir, oysa palindrom karşılaştırması özgün `n`'yi ister. Satır 21: n = 0 için döngü hiç çalışmaz, ters 0 kalır ve 0 doğru biçimde palindrom sayılır. Satır 23: "basamaklardan sayı inşası" deseni; H7'de ikili sayıya dönüşümde tersi gerekecek.

Tablo 6.6
Örnek 10 izleme tablosu (n = 2026)

Tur	sayi (başta)	son	ters (sonra)	sayi (sonra)
1	2026	6	6	202
2	202	2	62	20
3	20	0	620	2
4	2	2	6202	0
5	0	—	6202	koşul yanlış → çık

7–8. Örnek çalıştırmalar ve test tablosu (son sütun: gerçek çıktı).

Tablo 6.7
Örnek 10 test tablosu (normal, sınır ve hatalı girdiler)

Test	Girdi	Beklenen	Gerçek	Sonuç
palindrom	12321	palindrom	Ters: 12321 · 12321 bir palindrom sayıdır.	✓
normal	2026	6202, değil	Ters: 6202 · 2026 palindrom değildir.	✓
sondaki 0	1230	321, değil	Ters: 321 · 1230 palindrom değildir.	✓
sınır: 0	0	0, palindrom	Ters: 0 · 0 bir palindrom sayıdır.	✓
tersi int'e sığmaz	1999999999	9999999991	Ters: 9999999991 · 1999999999 palindrom değildir.	✓
geçersiz	-5	reddedilir	Geçersiz girdi.	✓

9. Değiştir–genişlet. (a) Sayının basamaklarından en büyüğünü bulun (Alıştırma 6.8). (b) Ters çevirmeden palindrom denetimi: ilk ve son basamağı karşılaştırıp ikisini birden atmak mümkün mü? (c) `ters`'i `int` yapıp 1999999999'u `-fsanitize=undefined` ile deneyin.

4.6 Klasik algoritma, girdi doğrulama ve oyun (Örnek 11–13)

Örnek 11 ★★ — Öklid algoritması ile EBOB ve EKOK

H2'de Öklid algoritmasını akış diyagramı ve izleme tablosuyla çözmüş, ilk C karşılığını (H02_12) görmüştük. Bu sürüm her adımı yazdırır ve EKOK'u ekler: EBOB(a, b) · EKOK(a, b) = a · b.

H06_11_ebob_ekok.c · satır 20–34

```
20      int x = a, y = b, adim = 0;
21      printf("adım      x      y  x %% y\n");
22      while (y != 0) {                  /* her turda y kesin olarak küçülür */
23          int r = x % y;
24          adim++;
25          printf("%4d %6d %6d %6d\n", adim, x, y, r);
26          x = y;
27          y = r;
28      }
29
30      int ebob = x;
31      /* önce böl, sonra çarp: ara sonuç küçük kalsın */
32      long long ekok = (long long)a / ebob * b;
33      printf("EBOB(%d, %d) = %d\n", a, b, ebob);
34      printf("EKOK(%d, %d) = %lld\n", a, b, ekok);
```

ÇIKTI

```
$ ./ebob_ekok      # Öklid'in klasik örneği
Pozitif iki tamsayı girin: 1071 462
adım      x      y  x % y
   1   1071    462    147
   2    462    147     21
   3    147     21      0
EBOB(1071, 462) = 21
EKOK(1071, 462) = 23562
```

Sonlanma. Her turda `y`'nin yerine `x % y` gelir ve kalan her zaman bölenden küçüktür: `y` kesin olarak küçülür, negatif olamaz, öyleyse en geç `y` = 0'da döngü biter. 18 48 sırasıyla girilirse ilk adım yalnızca yer değiştirir (18 % 48 = 18), bir tur fazla sürer. **Taşma.** `a * b / ebob` yazsaydık `a * b` `int`'e sığmayabilirdi; önce bölüp sonra çarpmak ara sonucu küçük tutar ve `long long` dönüşümü çarpımı güvenceye alır.

Örnek 12 ★★ — Geçerli girdi alınana kadar tekrar

Problem. 0–100 aralığında bir puan alınana kadar kullanıcıya yeniden sorun; harf girilirse, aralık dışı değer girilirse ya da girdi kapanırsa (EOF) doğru davranın. H5'te (S3) tek bir `if` ile programı sonlandırıyorduk; `do-while` kullanıcıya yeniden şans verir.

H06_12_gecerli_girdi.c · satır 16–34

```
16      do {
17          printf("Puan girin (0-100): ");
18          okunan = scanf("%d", &puan);
19
20          if (okunan == EOF) {          /* girdi akışı kapandı */
21              printf("\nGirdi sona erdi.\n");
```

```
            return 1;
        }
        if (okunan == 0) {              /* sayı değil: karakterler tamponda */
            printf("  Sayı bekleniyordu.\n");
            int c;
            while ((c = getchar()) != '\n' && c != EOF) {
                /* bilinçli boş gövde: satırın kalanını at */
            }
            puan = -1;                  /* geçersiz say: döngü sürsün */
        } else if (puan < 0 || puan > 100) {
            printf("  %d aralık dışında.\n", puan);
        }
    } while (puan < 0 || puan > 100);
```

ÇIKTI

```
$ ./gecerli_girdi      # geçersizler, sonra geçerli
Puan girin (0-100): abc
  Sayı bekleniyordu.
Puan girin (0-100): 150
  150 aralık dışında.
Puan girin (0-100): -5
  -5 aralık dışında.
Puan girin (0-100): 85
Kabul edilen puan: 85
```

Gözlemler. EOF (Ctrl+D) testinde program “Girdi sona erdi.” yazıp 1 ile çıkar (ciktilar dosyası). Gövde en az bir kez çalışmalıdır (önce sorulur): `do-while`. Döngü koşulu, S2’deki De Morgan dönüşümünün ta kendisidir. Harf girildiğinde `puan` = -1 yapılarak döngünün sürmesi güvenceye alınır. Bir sınır: `12abc` girildiğinde `scanf` 12’yi okur ve 1 döndürür; `abc` tamponda kalır, program 12’yi kabul eder. Bunu da reddetmek için sayıdan sonraki karakterin `'\n'` olduğu denetlenebilir (H13’te satır okuma).

Örnek 13 ★★ — Sayı tahmin oyunu

Problem. Program 1–100 arasında gizli bir sayı tutar; kullanıcı en çok 7 denemede bulmaya çalışır, her yanlış tahminde “daha büyük/küçük” ipucu alır. Yeni ögeler: `rand`/`srand`, deneme **sayacı**, `bildi` **bayrağı**, bulununca `break`, geçersiz girdide hak yakmadan `continue`.

H06_13_sayi_tahmin.c · satır 39–51

```
            printf("  Lütfen bir sayı girin.\n");
            continue;                   /* hak düşmeden yeniden sor */
        }
        deneme++;
        if (tahmin == gizli) {
            bildi = 1;
            break;                      /* bulundu: döngüden hemen çık */
        } else if (tahmin < gizli) {
            printf("  Daha BÜYÜK bir sayı deneyin.\n");
        } else {
            printf("  Daha KÜÇÜK bir sayı deneyin.\n");
        }
    }
```

ÇIKTI (GERÇEK OTURUMDAN KESİT; TAHMİNLER 50, 25, 37, 43, 40, 38)

```
$ ./sayi_tahmin      # kazanma
1-100 arasında bir sayı tuttum. 7 hakkınız var.
Tahmin 1: 50
  Daha KÜÇÜK bir sayı deneyin.
…
Tahmin 6: 38
Tebrikler! 6 denemede bildiniz.
```

Kayıtlı oturumlar `-DSABIT_TOHUM=2026` ile derlenmiştir; bu tohumla gizli sayı 38'dir (ikinci oturum, `kodlar/ciktilar/H06_13_sayi_tahmin.txt`). Öğrenci sürümü tohumu saatten alır. Her ipucu aralığı yaklaşık yarıya indirdiği için akıllı bir oyuncu 1–100'de en çok 7 tahminde bulur ($2^7 = 128 \geq 100$); 7 hak bu yüzden seçildi (genişletmesi: Bölüm 8, eşli programlama).

4.7 Örnek 14 ★★★ — Collatz dizisi ve adım sayısı

Problem. n çiftse n/2, tekse 3n + 1 yapılır; n = 1 olunca durulur. Adım sayısını ve dizinin ulaştığı en büyük değeri (tepe) bulun. Kural son derece basittir ama döngünün **her başlangıç değeri için biteceği kanıtlanmamıştır** (Bölüm 10, Merak Kutusu).

`H06_14_collatz.c · satır 24–33`

```
24      while (n != 1) {                    /* biteceği kanıtlanmış değil! */
25          if (n % 2 == 0) {
26              n = n / 2;
27          } else if (n <= (LLONG_MAX - 1) / 3) {
28              n = 3 * n + 1;
29          } else {                        /* 3n + 1 long long'a sığmaz */
30              printf("\nTaşma riski: durduruldu.\n");
31              return 1;
32          }
33          adim++;
```

Gerçek çıktı (n = 6): "6 3 10 5 16 8 4 2 1 → 8 adımda 1'e ulaşıldı; en büyük değer: 16".

27 gibi küçük bir başlangıç değeri **111 adım** sürer ve **9 232**'ye kadar tırmanır (gerçek çıktının tamamı dosyada). 3n + 1 işlemi `long long`'u taşırabileceği için hesaplamadan önce sınır denetlenir; güvenli olmayan bir adımda program durur.

5. Hata Avcısı

Döngü hataları iki türlüdür: program hiç bitmez (sonsuz döngü) ya da biter ama yanlış sayıda tur yapar. Sonsuz döngülerin **saf** hâlleri derslikte çalıştırılıp Ctrl+C ile durdurulmak içindir; kayıtlı çıktıları `timeout 2` ile alınmıştır (2 saniye sonra durdurulur, çıkış kodu **124**). Notta gösterilen döngü çıktıları, aynı hatayı taşıyan ama bir **gösterim tur sınırı** eklenmiş sürümlerdendir; sınır hatanın parçası değildir. Tüm dosyalar `kodlar/hata_avcisi/` altında, düzeltilmiş sürümleriyle birliktedir.

Tablo 6.8
Haftanın Hata Avcısı özeti (tüm iletiler ve çıktılar gerçektir)

#	Hata	Belirti / ileti	Neden	Çözüm
HA1	Güncelleme unutulmuş	Program bitmez; 2 saniyede 98 353 152 bayt "1 " yazdı	Koşuldaki değişken hiç değişmiyor	Her tur sona yaklaştırsın (`i++`)

#	Hata	Belirti / ileti	Neden	Çözüm
HA2	`for (…);` gövdesiz	GCC ve Clang uyarır; toplam 6	`;` boş gövdedir	`;` yok, `{ }` gövde
HA3	Bir-fark (`<` / `<=`)	1..10 toplamı 45 (9 tur)	Sınır dahil edilmedi	Uç değerleri izle: `i <= N`
HA4	Toplayıcı sıfırlanıyor	Toplam = son not	Her turda yeniden 0	Başlatma döngüden önce
HA5	Kayan noktalı sayaç	`x != 1.0` hiç yanlış olmaz	0,1 ikilide tam yazılamaz (H3)	Tamsayı sayaç; `x = i / 10.0`
HA6	scanf'e harf	İstem sonsuza dek yinelenir	0 döner, harfler tamponda kalır	Dönüş değeri + `getchar` temizleme
HA7	Zune döngüsü	31.12.2008'de sonsuz döngü	gun = 366'da hiçbir şey değişmiyor	Her tur ya ilerle ya `break`

HA1 · Güncellemesi unutulmuş döngü ve HA2 · for (…); tuzağı

hata_avcisi/H06_HA1_guncelleme_yok.c · satır 14–17

```
14      while (i <= 5) {
15          printf("%d ", i);
16          /* i++ unutuldu: koşul hep doğru kalır */
17      }
```

HA1: SAF SÜRÜM, TİMEOUT 2 İLE — GERÇEK

```
$ timeout 2 ./ha1 > H06_HA1_guncelleme_yok_tam_cikti.txt; echo "çıkış kodu: $?"
çıkış kodu: 124
$ wc -c < H06_HA1_guncelleme_yok_tam_cikti.txt
98353152
```

2 saniyede 98 353 152 bayt, yani on milyonlarca "1 " yazıldı: bilgisayar yorulmadan ve **anlamsızca** tekrar etti. Gösterim sürümü 20 turdan sonra "20 tur geçti, i hâlâ 1: döngü bitmedi -> sonsuz döngü!" yazıp durur. Düzeltme: gövdeye `i++` eklemek. HA2'de ise döngü başlığının sonundaki noktalı virgül döngünün **boş gövdesidir**:

hata_avcisi/H06_HA2_govdesiz_for.c · satır 13–16

```
13      for (i = 1; i <= 5; i++);
14      {
15          toplam += i;
16      }
```

HA2: GCC UYARISI (İLK SATIRI) VE ÇALIŞTIRMA — GERÇEK

```
$ gcc -std=c17 -Wall -Wextra -pedantic -g H06_HA2_govdesiz_for.c -o ha2
H06_HA2_govdesiz_for.c:13:5: warning: this ‘for’ clause does not guard... [-
Wmisleading-indentation]
…
$ ./ha2
1..5 toplamı: 6
```

Döngü beş kez "hiçbir şey" yapar, `i` 6 olur; alttaki blok döngüden sonra **bir kez** çalışır ve toplam 6 çıkar. GCC 13 `-Wall` içindeki `-Wmisleading-indentation` ile, Clang 18 "for loop has empty body [-Wempty-body]" iletisiyle uyarır. Derleme başarılı olsa bile uyarıyı okuyun.

HA3 · Bir-fark hatası ve HA4 · sıfırlanan toplayıcı

`H06_HA3_bir_fark.c`'de `for (int i = 1; i < N; i++)` (N = 10) "1..10 toplamı: 45 (9 tur)" yazar; düzeltilmiş `i <= N` sürümü "1..10 toplamı: 55 (10 tur)". Derleyici bu hatayı göremez; korunmanın yolu **uç değerleri izlemektir**: ilk ve son tur hangi değerle çalışıyor, kaç tur? ("10 metrelik çite metrede bir direk: 11 direk"). `H06_HA4_toplayici_sifirlama.c` ise `toplam = 0;` satırını döngünün **içine** koyar; 70, 80, 90, 100 girdisiyle "Toplam: 100, ortalama: 25.00" yazar, düzeltilmiş sürüm "Toplam: 340, ortalama: 85.00". Başlatma döngüden **önce**, bir kez yapılır.

HA5 · Kayan noktalı sayaç ve HA6 · scanf'e harf

hata_avcisi/H06_HA5_kayan_sayac.c · satır 13–15

```
13      for (double x = 0.0; x != 1.0; x += 0.1) {
14          printf("x = %.17f\n", x);
15      }
```

HA5 GÖSTERİM SÜRÜMÜ (14 TUR SINIRIYLA; GERÇEK ÇIKTIDAN KESİT)

```
$ ./ha5_gosterim
x = 0.00000000000000000
…
x = 0.89999999999999991
x = 0.99999999999999989
x = 1.09999999999999987
14 tur geçti, x 1.0'ı atladı: döngü bitmedi -> sonsuz döngü!
Döngü bitti.
```

0,1 ikili tabanda sonlu yazılamadığı için (H3) her toplama küçük bir yuvarlama hatası ekler: 10 turdan sonra `x` = 0,99999999999999989'dur, `x != 1.0` doğru kalır ve sonraki tur 1,0'ı **atlar**. Saf sürüm 2 saniyede 138 842 112 bayt yazdı. Düzeltme: sayacı tamsayı tutup `x = i / 10.0` ile her turda yeniden hesaplamak (`<` kullanmak da yetmez: `while (x < 1.0)` 10 değil **11** tur döner, Q32).

hata_avcisi/H06_HA6_scanf_harf.c · satır 14–17

```
14      do {
15          printf("Puan (0-100): ");
16          scanf("%d", &puan);             /* dönüş değeri yok sayılıyor! */
17      } while (puan < 0 || puan > 100);
```

HA6 GÖSTERİM SÜRÜMÜ (SCANF DÖNÜŞÜ YAZDIRILIR, 5 TUR SINIRIYLA)

```
$ ./ha6_gosterim
Puan (0-100): abc
[scanf 0 döndürdü, puan = -1]
Puan (0-100): [scanf 0 döndürdü, puan = -1]
Puan (0-100): [scanf 0 döndürdü, puan = -1]
Puan (0-100): [scanf 0 döndürdü, puan = -1]
Puan (0-100): [scanf 0 döndürdü, puan = -1]
5 tur geçti: 'abc' hâlâ tamponda -> sonsuz döngü!
Puan: -1
```

Kullanıcı yalnızca bir kez "abc" yazdı; sonraki istemlerde yazma fırsatı bile olmadı, çünkü `scanf` her turda tampondaki aynı `a`'ya takılıp 0 döndürdü. Saf sürüm 2 saniyede 368 156 672 bayt istem yazdı. Düzeltme: dönüş değerini denetle, 0 ise satırın kalanını at (Örnek 12).

HA7 · Zune döngüsü (gerçek dünyadan)

Bölüm 1'deki döngünün saf sürümü 10593. gün için hiç bitmez (`timeout 2` ile: "Gün 10593 için yıl hesaplanıyor… · çıkış kodu: 124"). Gün sayısını okuyan gösterim sürümü 30 Aralık 2008 (10592) için "Yıl 2008, yılın 365. günü", 1 Ocak 2009 (10594) için "Yıl 2009, yılın 1. günü" yazar; 31 Aralık'ta ise:

GÖSTERİM SÜRÜMÜ, 10593 (GERÇEK ÇIKTIDAN KESİT)

```
$ ./ha7_gosterim      # 31 Aralık 2008
1.1.1980'den beri geçen gün (1 = 1.1.1980): 10593
tur 27: yil = 2007, gun = 731
tur 28: yil = 2008, gun = 366
tur 29: yil = 2008, gun = 366
…
tur 40: yil = 2008, gun = 366
40 tur geçti, gun hâlâ 366: döngü bitmedi -> sonsuz döngü!
```

Yalnızca bir gün, artık yılın son günü döngüyü kilitliyor: bu yüzden hata 2008 boyunca görülmedi ve ertesi gün kendiliğinden "düzeldi". Düzeltilmiş sürüm (`H06_HA7_zune_dongusu_duzeltilmis.c`) her turda yılın uzunluğunu `artik ? 366 : 365` ile hesaplar, gün bu yılın içindeyse `break` ile çıkar, değilse günü o uzunluk kadar azaltır: her tur ya çıkar ya da en az 365 gün ilerler. Düzeltilmiş sürüm 10593 için "Yıl 2008, yılın 366. günü", 366 (31.12.1980) için "Yıl 1980, yılın 366. günü" yazar (gerçek çıktılar).

6. Mühendis Gözüyle: Her Döngü Bitmeli ve Doğru Sayıda Dönmeli

- **Sonlanma bir tasarım kararıdır.** Her döngüde "hangi nicelik her turda küçülüyor?" sorusunu yanıtlayın (Bölüm 3.5); yanıt yoksa ya da girdiye bağlıysa bir **üst sınır** koyun (Örnek 13'te 7 hak). Gömülü sistemlerde son savunma, yazılım belirli aralıklarla sinyal vermezse sistemi yeniden başlatan **bekçi zamanlayıcılarıdır** (watchdog timer).
- **Test, uç değerlerde yapılır: 0 tur** (N = 0, ilk girdi nöbetçi), **1 tur**, **çok tur** ve sınırın hemen öncesi/sonrası (Zune'da 365, 366, 367. gün). **Taşmayı öngörün:** biriktiricinin tipini en büyük sonuca göre seçin (Örnek 2'de N ≤ 65 535, Örnek 5'te 20!), sınırı işlemden **önce** denetleyin.
- **Girdiye güvenmeyin.** `scanf` dönüşü denetlenmeyen bir döngü, tek bir harfle sonsuz döngüye sokulabilir (HA6). Dışarıdan gelen verinin bir programı kilitleyebilmesi bir **hizmet engelleme** (denial of service) açığıdır.

7. Mini-OBS v0.6: N Öğrenci, Akış Hâlinde

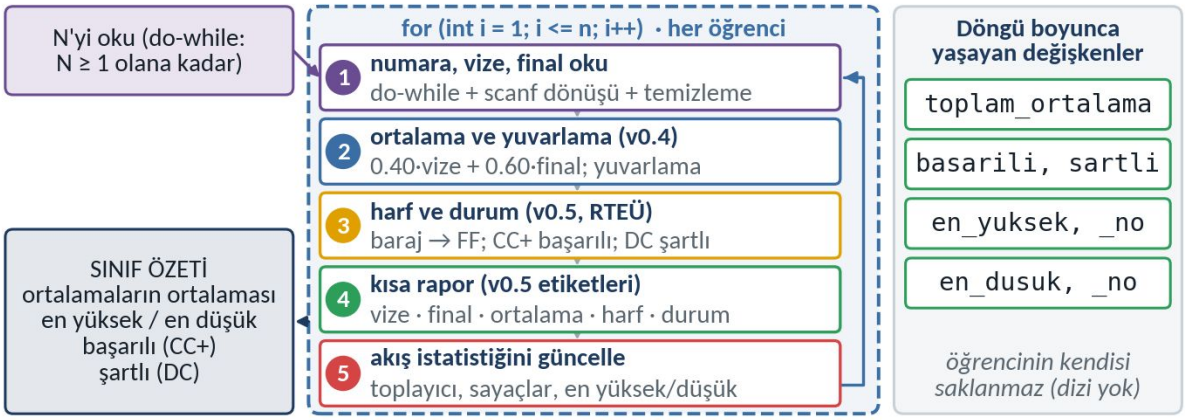
v0.5'e kadar Mini-OBS tek öğrencinin notlarını işliyordu. **v0.6:** N öğrenci için döngüyle not girişi ve sınıf özeti. Dizi yok (H10): her öğrenci okunur, işlenir ve "unutulur"; yalnızca özet değişkenleri kalır (Şekil 6.7). Kaynak: `91_Proje_MiniOBS/v0.6/` (kopyası `kodlar/proje/`).

Gereksinimler. (1) N ≥ 1; geçersizse `do-while` ile yeniden sorulur. (2) Her öğrenci için numara (pozitif), vize ve final (0–100); sayı olmayan ya da aralık dışı girdi `scanf` dönüşü + tampon temizleme ile yeniden sorulur. (3) v0.5 etiketleriyle kısa rapor (Vize (%40), Final (%60), ortalama, yuvarlanmış, harf, durum). (4) Sınıf özeti: sınıf ortalaması, en yüksek/en düşük (numarasıyla), **başarılı (CC+)** ve **şartlı (DC)** sayısı / N. **Kurallar** v0.5'ten aynen (RTEÜ yönetmeliği, Madde 30): %40 vize + %60 final, `(int)(ortalama + 0.5)` (ders içi modelleme varsayımı); final < 50 → FF; harf aralıkları ve durumlar H5'teki tablodaki gibidir (CC ve üstü BAŞARILI, DC ŞARTLI BAŞARILI, DD–FF BAŞARISIZ).

proje/mini_obs_v0_6.c · satır 155–163

```
155         } else if (yuvarlanmis >= GECME_ORTALAMA) {
156             printf("BAŞARILI\n");
157             basarili++;                              /* sayaç */
158         } else if (yuvarlanmis >= SARTLI_ORTALAMA) {
159             printf("ŞARTLI BAŞARILI (YANO ≥ 2,00 ise)\n");
160             sartli++;                                /* sayaç */
161         } else {
162             printf("BAŞARISIZ\n");
163         }
```

Şekil 6.7
Mini-OBS v0.6: sayaç kontrollü dış döngü, girdi doğrulama döngüleri ve akış istatistikleri



Sınıf özeti bu haftanın desenleriyle, akış hâlinde hesaplanır: `toplam_ortalama += ortalama` (toplayıcı), `basarili++` ve `sartli++` (sayaçlar), `i == 1 || ortalama > en_yuksek` ile "şimdiye kadarki en iyi". Numara, vize ve final için neredeyse aynı `do-while` bloğu üç kez yazıldı: DRY ilkesine aykırı. H8'de bu blok bir fonksiyona dönüşecek (v0.8). **Kabul ölçütleri:** sıfır uyarı; T1–T5 testlerinin çıktıları aşağıdakilerle aynı; geçersiz girdilerde program çökmez ve sonsuz döngüye girmez; girdi yarıda biterse (EOF) "Girdi sona erdi." ile çıkar.

Tablo 6.9
Mini-OBS v0.6 test senaryoları (girdi dosyası ile gerçek çalıştırma)

Test	Senaryo	Sınıf ort.	En yüksek (no)	Başarılı	Şartlı	Çıkış
T1	5 öğrencili normal sınıf	66.48	93.00 (2301010047)	3 / 5	1 / 5	0
T2	geçersiz girdiler içeren sınıf	76.97	84.00 (2301010050)	1 / 2	0 / 2	0
T3	tek öğrenci	100.00	100.00 (2301010099)	1 / 1	0 / 1	0
T4	sınır değerler: final tam 50; 59.5 → 60 (CC); 49.6 → 50 (DC); 89.5 → 90 (AA)	64.65	89.50 (2301010063)	3 / 4	1 / 4	0
T5	girdi dosyası yarıda biter (EOF)	—	—	—	—	1

8. Alıştırmalar ve Atölye

Atölye. (1) **Desen eşleştirme** (15 dk): "sınıfta 50'nin altında kaç not var?", "basamakların çarpımı", "şifrede rakam var mı?", "en erken teslim saati", "1..N'nin karelerinin toplamı" problemlerini

desenlerle ve başlangıç değerleriyle eşleştirin. (2) **PRIMM** (20 dk; Sentance vd., 2019): Örnek 10 üzerinde tahmin et, çalıştır, incele (izleme tablosu), değiştir, yarat (Alıştırma 6.8). (3) **Eşli programlama** (25 dk; sürücü–gözlemci): sayı tahmin oyununa en iyi skor ve zorluk düzeyi ekleyin. Dosyalar: `kodlar/alistirmalar/` (boşluklu ve Parsons dosyaları bilerek derlenmez).

Alıştırma 6.1 ★☆☆ *Çıktı tahmini*

Çıktıyı çalıştırmadan tahmin edin; her döngü için izleme tablosu çizin (`i`'nin son değeri?).

alistirmalar/H06_A01_cikti_tahmini.c · satır 11–23

```c
    int i = 20;
    while (i > 0) {
        printf("%d ", i);
        i -= 6;
    }
    printf("| i = %d\n", i);

    int j = 5;
    do {
        printf("[%d]", j);
        j += 5;
    } while (j < 5);
    printf(" j = %d\n", j);
```

Alıştırma 6.2 ★★☆ *Çıktı tahmini*

`break` ve `continue` birlikte. Hangi satırlar yazılır, `toplam` son olarak kaçtır?

alistirmalar/H06_A02_break_continue_tahmin.c · satır 11–22

```c
    int toplam = 0;
    for (int i = 1; i <= 10; i++) {
        if (i % 2 == 0) {
            continue;
        }
        if (toplam > 10) {
            break;
        }
        toplam += i;
        printf("i = %d, toplam = %d\n", i, toplam);
    }
    printf("son: %d\n", toplam);
```

Alıştırma 6.3 ★★☆ *Hata bul*

N sayının ortalamasını hesaplaması gereken programda **üç** hata var: biri derleyici uyarısıyla, ikisi yalnızca izleme tablosuyla görünür.

alistirmalar/H06_A03_hata_bul_ortalama.c · satır 16–24

```c
    int toplam = 0;
    for (int i = 0; i <= n; i++) {
        printf("%d. sayı: ", i + 1);
        if (scanf("%d", &x) != 1) {
            return 1;
        }
        toplam = x;
    }
    printf("Ortalama: %.2f\n", toplam / n);
```

Alıştırma 6.4 ★★☆ *Hata bul*

Nöbetçi döngü `70 80 -1` girdisiyle "3 not" ve yanlış ortalama yazıyor. Neden? Öncül okumayla düzeltin; hiç not girilmezse ne olur?

alistirmalar/H06_A04_hata_bul_nobetci.c · satır 10–19

```
10      int not_degeri = 0, toplam = 0, sayac = 0;
11      while (not_degeri != -1) {
12          printf("Not (-1 bitirir): ");
13          if (scanf("%d", &not_degeri) != 1) {
14              break;
15          }
16          toplam += not_degeri;
17          sayac++;
18      }
19      printf("%d not, ortalama %.2f\n", sayac, (double)toplam / sayac);
```

Alıştırma 6.5 ★★☆ *Parsons*

Satırları, 4071'in basamaklarını sondan başa yazdırıp (1704) basamak sayısını (4) bulan programa dönüşecek biçimde sıralayın ve girintileyin; bir satır fazladır.

alistirmalar/H06_A05_parsons_basamak.c · satır 11–19

```
11      } while (sayi != 0);
12      printf("\n%d basamak\n", basamak);
13      sayi /= 10;
14      int sayi = 4071;
15      sayi %= 10;
16      do {
17      printf("%d", sayi % 10);
18      int basamak = 0;
19      basamak++;
```

Alıştırma 6.6 ★★☆ *Kod tamamla*

Boşlukları doldurun: 1..N arasında 3'ün **ya da** 5'in katlarının toplamı ve adedi. N = 20 için beklenen: `adet = 9, toplam = 98`.

alistirmalar/H06_A06_kod_tamamla_katlar.c · satır 15–21

```
15      int toplam = ____, adet = ____;
16      for (int i = 1; ____; i++) {
17          if (____) {
18              toplam += i;
19              ____;
20          }
21      }
```

Alıştırma 6.7 ★☆☆ *Kod yaz*

Kullanıcı 0 girene kadar sayı okuyan, negatifleri `continue` ile atlayıp pozitiflerin adedini ve toplamını yazan programı yazın. Test: `5 -3 8 0` → 2 pozitif, toplam 13; ilk girdi 0 → 0 pozitif.

Alıştırma 6.8 ★★☆ *Kod yaz*

Negatif olmayan bir tamsayının en büyük basamağını ve bu basamağın kaç kez geçtiğini bulun (38783 → 8, 2 kez; 0 → 0, 1 kez). Hangi üç deseni birleştirdiniz?

Alıştırma 6.9 ★★☆ *Kod yaz*

Yalnızca çıkarma kullanarak a / b ve a % b'yi hesaplayın ve tur sayısını yazın (17 5 → 3, kalan 2). Döngü değişmezini yazın: her turun başında `a == b * bolum + kalan`. Döngü neden biter?

Alıştırma 6.10 ★★★ *Kod yaz*

Luhn denetimi. Kart numaralarında sağdan her ikinci basamak 2 ile çarpılır (9'dan büyükse 9 çıkarılır); basamak toplamı 10'a bölünüyorsa numara geçerlidir. `long long` bir numarayı denetleyin. Test: 79927398713 geçerli, 79927398710 geçersiz. Hangi bayrak her basamakta tersine döner?

Alıştırma 6.11 ★★★ *Kod yaz*

Tersine tahmin oyunu. Kullanıcı 1–100 arasında bir sayı tutar; program tahmin eder, kullanıcı `b` (daha büyük), `k` (daha küçük) ya da `d` (doğru) yazar. Program en çok 7 denemede bulmalı, tutarsız yanıtları fark etmeli. İpucu: alt/üst sınır tutun, ortayı tahmin edin; `scanf(" %c", &yanit)` (H3).

9. Kendini Değerlendirme

Yapabiliyorum…	
☐ while, do-while ve for'un çalışma sırasını anlatıp birini diğerine dönüştürebiliyorum.	☐ Bir problemin sayaç ya da nöbetçi değer kontrollü olduğuna karar verebiliyorum.
☐ Altı döngü desenini doğru başlangıç değeriyle yazabiliyorum.	☐ Bir döngü için izleme tablosu çizip son (yanlış) denetimi gösterebiliyorum.
☐ Döngünün neden biteceğini (ilerleme ölçüsü) ve neden doğru olduğunu (değişmez) açıklayabiliyorum.	☐ Sonsuz döngü ve bir-fark hatalarını bulabiliyor, güvenli girdi döngüsü yazabiliyorum.
☐ Döngünün assembly'de geriye doğru koşullu atlama olduğunu gösterebiliyorum.	☐ rand/srand ile belirli aralıkta sayı üretip tohumun rolünü açıklayabiliyorum.

10. AP-2 Köprüsü ve Merak Kutusu

AP-2 KÖPRÜSÜ · Döngü mü, formül mü? Büyüme hızına ilk bakış

- 1 + … + N'yi döngü **N adımda**, formül N ne olursa olsun **sabit sayıda** işlemde bulur; N = 10^9'da fark saniyeler ile milisaniyeler arasındadır (Bölüm 3.7). "Girdi büyüdükçe adım sayısı nasıl büyüyor?" sorusu AP-2'de algoritma çözümlemesinin temelidir (burada biçimsel gösterime girmiyoruz).
- "Şimdiye kadarki en iyi" deseni (Örnek 8) AP-2'de **seçme sıralamasının** iç döngüsüdür; döngü değişmezi (Bölüm 3.5) sıralama ve aramanın **doğruluğunu** gerekçelendirir.

> **MERAK KUTUSU · Collatz sanısı: bitip bitmediğini bilmediğimiz döngü**
>
> Örnek 14'teki kural, 1930'larda Lothar Collatz'a atfedilen bir soruya dayanır: hangi pozitif tamsayıdan başlanırsa başlansın dizi 1'e ulaşır mı? Sanı çok büyük başlangıç değerleri için bilgisayarla doğrulanmıştır ama **kanıtlanamamıştır**; matematiğin ünlü açık problemlerinden biridir (Lagarias, 1985). Yani `while (n != 1)` için bir ilerleme ölçüsü bilinmiyor: 27'den başlayan dizi 111 adımda 9 232'ye kadar yükselip düşer (webde Collatz görselleştiricisi). Döngünün biteceğini "her denediğimizde bitti" diyerek değil, ancak bir **kanıtla** bilebiliriz.

11. Özet, Kavram Haritası ve Terim Sözlüğü

Özet. Döngü = başlatma + koşul + gövde + güncelleme (Şekil 6.8). Desenler doğru başlangıç değeriyle kurulur. Her döngü için iki soru: bitecek mi (ilerleme ölçüsü), doğru mu (değişmez, izleme tablosu, uç değer testi)? Makine düzeyinde döngü, geriye doğru koşullu atlamadır.

Şekil 6.8
Haftanın kavram haritası



Büyük Sorunun yanıtı. Bilgisayar aynı işi milyarlarca kez nasıl yorulmadan yapar? Çünkü döngü, işlemcinin en temel iki komutuna indirgenir: karşılaştır ve geriye atla. Bu makinede iyileştirmesiz kod 1 milyar turu 2,5 saniyede, gcc -O2 0,33 saniyede tamamladı. **Ne zaman durması gerektiğini nereden bilir?** Bilmez: onu biz söyleriz. Koşul ve her turda sona yaklaştıran güncelleme doğru kurulmuşsa döngü biter; Zune'daki gibi tek bir günde ilerleme durursa aynı yorulmazlıkla sonsuza dek döner. Bu yüzden her döngüde "neden bitecek?" diye soruyor, sınır değerlerde test ediyoruz.

Terim sözlüğü

Tablo 6.10
Haftanın terimleri (yol haritası Bölüm 6.8 ile uyumlu)

Türkçe	İngilizce	Kısa açıklama
döngü; tur (yineleme)	loop; iteration	Tekrarlanan deyimler; döngünün bir kez çalışması
sayaç kontrollü döngü	counter-controlled loop	Tur sayısı döngüden önce belli
nöbetçi değer	sentinel value	Verinin bittiğini bildiren, veri olamayacak değer
öncül okuma	priming read	Nöbetçi döngüde ilk değerin döngüden önce okunması
sonsuz döngü	infinite loop	Koşulu hiç yanlış olmayan döngü
bir-fark hatası	off-by-one error	Döngünün bir tur fazla ya da eksik çalışması

Türkçe	İngilizce	Kısa açıklama
döngü değişmezi	loop invariant	Her turun başında doğru olan önerme
toplayıcı; sayaç; bayrak	accumulator; counter; flag	Temel döngü desenlerinin değişkenleri
tampon; dosya sonu	buffer; end-of-file (EOF)	Okunmayı bekleyen karakterler; girdinin bitişi
sözde rastgele sayı; tohum	pseudo-random number; seed	Belirlenimci üreteç; dizinin başlangıç değeri
koşullu atlama; yazmaç	conditional jump; register	cmp + jle/jne; işlemcideki hızlı bellek
kapalı form; iyileştirme	closed form; optimization	Döngüsüz formül; derleyicinin kodu hızlandırması

Gelecek hafta: Döngüler II ve ara proje. H7'de döngüleri **iç içe** kuracağız (yıldız desenleri, asallık testi ve √n, `clock()` ile süre ölçümü); Mini-OBS v0.7'de `do-while` + `switch` ile **menü güdümlü ana döngü** kurulacak ve ara proje olarak bir ATM benzetimi yazacaksınız. Hazırlık: desen kartlarını tekrar edin, Alıştırma 6.9'u tamamlayın.

12. Kaynaklar

Bryant, R. E., & O'Hallaron, D. R. (2016). *Computer systems: A programmer's perspective* (3rd ed.). Pearson.

Coldewey, D. (2008, December 31). The day the Zunes stood still: Update. *TechCrunch*. https://techcrunch.com/2008/12/31/the-day-the-zunes-stood-still-update/

Deitel, P., & Deitel, H. (2022). *C how to program: With case studies introducing applications and systems programming* (9th ed., global ed.). Pearson.

Hayes, B. (2006). Gauss's day of reckoning. *American Scientist*, *94*(3), 200–204. https://www.americanscientist.org/article/gausss-day-of-reckoning

Hayes, B. (2009). *The Zune bug* [Blog post]. bit-player. http://bit-player.org/2009/the-zune-bug/

ISO/IEC. (2018). *Information technology — Programming languages — C* (ISO/IEC Standard No. 9899:2018). International Organization for Standardization.

Kernighan, B. W., & Ritchie, D. M. (1988). *The C programming language* (2nd ed.). Prentice Hall.

King, K. N. (2008). *C programming: A modern approach* (2nd ed.). W. W. Norton.

Lagarias, J. C. (1985). The 3x + 1 problem and its generalizations. *The American Mathematical Monthly*, *92*(1), 3–23. https://doi.org/10.1080/00029890.1985.11971528

Microsoft says Zune players working again. (2009). *NBC News*. https://www.nbcnews.com/id/wbna28449091

Recep Tayyip Erdoğan Üniversitesi. (2025). *Recep Tayyip Erdoğan Üniversitesi ön lisans ve lisans eğitim-öğretim ve sınav yönetmeliği* (Resmî Gazete, 14 Aralık 2025, Sayı: 33107; son değişiklik: Resmî Gazete, 27 Şubat 2026, Sayı: 33181).

Sentance, S., Waite, J., & Kallia, M. (2019). Teaching computer programming with PRIMM: A sociocultural perspective. *Computer Science Education*, *29*(2–3), 136–176. https://doi.org/10.1080/08993408.2019.1608781